This strategy function combines the RSI calculation with trading logic to generate position signals. By encapsulating each component in its own function, we've created a modular, readable system that can be easily tested and refined.

Creating a Function Library: Your Trading Toolkit

As you develop more functions, you'll want to organize them into a reusable library. This approach is used by all professional trading firms to maintain and leverage their intellectual property.
Here's how you might organize a basic trading function library:

```
# indicators.py - Technical indicator functions

def calculate_sma(prices, period):
    """Calculate Simple Moving Average"""
    return sum(prices[-period:]) / period

def calculate_ema(prices, period, smoothing=2):
    """Calculate Exponential Moving Average"""
    ema = [sum(prices[:period]) / period]  # Initialize with SMA

    multiplier = smoothing / (period + 1)

    for price in prices[period:]:
        ema.append((price - ema[-1]) * multiplier + ema[-1])

    return ema[-1]

def calculate_rsi(prices, period=14):
    """Calculate Relative Strength Index"""
    # Implementation as shown earlier
    pass

# signals.py - Trading signal functions

def check_golden_cross(prices, fast_period=50, slow_period=200):
```